commons-cli-scope.txt
TXT 149.65KB

You are a code reviewer. The attached file "commons-cli-scope.txt" contains the Java source code of the Apache Commons CLI library (34 files, comments removed). Each file starts with a line "// ===== FILE: <path> =====".

Analyze the code for exactly these three code smells and ignore all others:

Code Smell Analysis: Apache Commons CLI

1. Long Method

Smell: Long Method
Code unit: `DefaultParser.handleShortAndLongOption(String)`
File: `org/apache/commons/cli/DefaultParser.java`
Evidence: A single method containing a large nested `if/else if/else` chain: `token.length() == 1` ; `pos == -1` with nested branches for `options.hasShortOption(token)` , `!getMatchingLongOptions(token).isEmpty()` , `getLongPrefix(token) != null` , `isJavaProperty(token)` , and finally `handleConcatenatedOptions` ; then an `else` branch with `opt.length() == 1` , `isJavaProperty(opt)` , and `handleLongOptionWithEqual` . It repeatedly calls `handleOption` , `processValue` , `stripLeadingAndTrailingQuotesDefaultOff` , and nulls out `currentOption` in each branch.
Explanation: The method handles short options, long options, `=` -embedded values, Java property prefixes, concatenated options, and argument parsing all in one body, making it long and hard to follow.

Smell: Long Method
Code unit: `DefaultParser.handleToken(String)`
File: `org/apache/commons/cli/DefaultParser.java`
Evidence: Branches on `skipParsing` , `OptionFormatter.DEFAULT_LONG_OPT_PREFIX.equals(token)` , `currentOption != null && currentOption.acceptsArg() && isArgument(token)` ,

`token.startsWith(DEFAULT_LONG_OPT_PREFIX)`, `token.startsWith(DEFAULT_OPT_PREFIX)`, else `handleUnknownToken`; then a trailing `if (currentOption != null && !currentOption.acceptsArg())` reset.

Explanation: This method orchestrates the full token-dispatch logic (skip mode, long/short/unknown handling, argument assignment, and state reset) in one place.

Smell: Long Method

Code unit: `HelpFormatter.appendOptions(A, int, Options, int, int)`

File: `org/apache/commons/cli/HelpFormatter.java`

Evidence: Contains `determineMaxSinceLength`, building `prefixList` and `max` in a loop over `optList`, computing `nextLineTabStop`, conditional `showSince` header construction with `createPadding(...)`, then a second loop over `optList` appending padding, `option.getSince()`, `deprecatedFormatFunction`, description, and calling `appendWrappedText`, plus `sb.append(getNewLine())`.

Explanation: The method performs option-prefix formatting, max-width computation, "since" column layout, header rendering, and wrapped description output all in one lengthy body.

Smell: Long Method

Code unit: `GnuParser.flatten(Options, String[], boolean)`

File: `org/apache/commons/cli/GnuParser.java`

Evidence: A loop over `arguments` with branches for `DEFAULT_LONG_OPT_PREFIX`, `DEFAULT_OPT_PREFIX`, `arg.startsWith(DEFAULT_OPT_PREFIX)` (nested `hasOption(opt)`, `equalPos` handling, `arg.substring(0,2)` handling), plus an inner `for (i++; i < arguments.length; i++)` loop when `eatTheRest`.

Explanation: The method handles long-option terminator, bare opt prefix, `=`-embedded options, two-character short options, and "eat the rest" propagation in a single long routine.

Smell: Long Method

Code unit: `PosixParser.flatten(Options, String[], boolean)`

File: `org/apache/commons/cli/PosixParser.java`

Evidence: A loop over tokens with branches for bare `-` / `--`, `startsWith(DEFAULT_LONG_OPT_PREFIX)` (with `indexOfEqual`, `getMatchingOptions`, `AmbiguousOptionException`), `startsWith(DEFAULT_OPT_PREFIX)` (with `burstToken`, `getMatchingOptions`, `AmbiguousOptionException`), else `processNonOptionToken`; plus `addRemaining(iter)`.

Explanation: The method performs all POSIX token classification, ambiguity detection, bursting, and remaining-token handling in one long body.

Smell: Long Method
Code unit: `CommandLine.getParsedOptionValues(Option, Supplier<T[]>)`
File: `org/apache/commons/cli/CommandLine.java`
Evidence: Null check on `option`, `get(defaultValue)`, cast to `Class<? extends T>`, `Array.newInstance(clazz, values.length)`, a `try` with a `for` loop over `values` calling `clazz.cast(option.getConverter().apply(values[i]))`, and `catch` wrapping via `ParseException.wrap(t)`.
Explanation: The method combines null handling, type extraction, array allocation, per-value conversion, and exception wrapping in a single body.

Smell: Long Method
Code unit: `DefaultParser.parse(Options, Properties, NonOptionAction, String...)`
File: `org/apache/commons/cli/DefaultParser.java`
Evidence: Sets `this.options`, `nonOptionAction`, resets `skipParsing` / `currentOption`, builds `expectedOpts`, resets every `optionGroup.setSelected(null)`, creates `cmd` via builder, loops `handleToken(argument)` over arguments, then calls `checkRequiredArgs()`, `handleProperties(properties)`, `checkRequiredOptions()`, returns `cmd`.
Explanation: The method performs parser initialization, option-group reset, token processing, property handling, and validation all in one sequence.

2. God Class

Smell: God Class
Code unit: `CommandLine`
File: `org/apache/commons/cli/CommandLine.java`
Evidence: Extremely large public API: `getArgList`, `getArgs`, `getOptionCount` (3 overloads), `getOptionObject` (2), `getOptionProperties` (2), `getOptions`, `getOptionValue` (~14 overloads for char/String/Option/OptionGroup/defaults/suppliers), `getOptionValues` (5), `getParsedOptionValue` (~15 overloads), `getParsedOptionValues` (~12 overloads), `hasOption` (4), `iterator`, `processPropertiesFromValues`, `resolveOption`, plus the nested `Builder` with `addArg`, `addOption`, `build`, `get`, `setDeprecatedHandler`.
Explanation: The class knows about argument storage, option lookup by char/String/Option/OptionGroup, typed parsing, properties extraction, deprecation handling, and its own builder — far too many responsibilities and heavy coupling to `Option`, `OptionGroup`, `Converter`, and `Properties`.

Smell: God Class

Code unit: `HelpFormatter`

File: `org/apache/commons/cli/HelpFormatter.java`

Evidence: Many public setters/getters (`setArgName` , `setDescPadding` , `setLeftPadding` , `setLongOptPrefix` , `setLongOptSeparator` , `setNewLine` , `setOptionComparator` , `setOptPrefix` , `setSyntaxPrefix` , `setWidth` and matching getters), a large set of `printHelp` overloads, `printOptions` , `printUsage` (2), `printWrapped` (2), protected `renderOptions` , `renderWrappedText` , `renderWrappedTextBlock` , `createPadding` , `findWrapPos` , `rtrim` , `appendOptions` , `appendOption` , `appendOptionGroup` , `appendWrappedText` , `determineMaxSinceLength` , `getDescription` , plus mutable public fields (`defaultWidth` , `defaultLeftPad` , `defaultDescPad` , etc.) and nested `Builder` / `OptionComparator` .

Explanation: It combines text-wrapping, padding, usage-syntax generation, option-group rendering, comparator configuration, deprecation formatting, and printing — too many responsibilities and extensive knowledge of `Options` / `Option` / `OptionGroup` internals.

Smell: God Class

Code unit: `DefaultParser`

File: `org/apache/commons/cli/DefaultParser.java`

Evidence: Fields `cmd` , `options` , `stopAtNonOption` , `nonOptionAction` , `currentToken` , `currentOption` , `skipParsing` , `expectedOpts` , `allowPartialMatching` , `stripLeadingAndTrailingQuotes` , `deprecatedHandler` ; methods `indexOfEqual` , `addArg` , `checkRequiredArgs` , `checkRequiredOptions` , `getLongPrefix` , `getMatchingLongOptions` , `handleConcatenatedOptions` , `handleLongOption` , `handleLongOptionWithEqual` , `handleLongOptionWithoutEqual` , `handleOption` , `handleProperties` , `handleShortAndLongOption` , `handleToken` , `handleUnknownToken` , `isArgument` , `isJavaProperty` , `isLongOption` , `isNegativeNumber` , `isOption` , `isShortOption` , `parse` (4 overloads), `stripLeadingAndTrailingQuotesDefaultOff/On` , `updateRequiredOptions` , plus nested `Builder` and `NonOptionAction` enum.

Explanation: The class holds all parser state and implements long-option matching, short-option bursting, property handling, required-option bookkeeping, ambiguity detection, and configuration — a large central class with many responsibilities.

Smell: God Class

Code unit: `Option`

File: `org/apache/commons/cli/Option.java`

Evidence: Fields for `argCount` , `argName` , `converter` , `deprecated` , `description` , `longOption` , `option` , `optionalArg` , `required` , `since` , `type` , `values` , `valueSeparator` ;

methods include `acceptsArg`, `add`, `addValue`, `clearValues`, `clone`, `equals`, `hashCode`, `getArgName`, `getArgs`, `getConverter`, `getDeprecated`, `getDescription`, `getId`, `getKey`, `getLongOpt`, `getOpt`, `getSince`, `getType`, `getValue` (3), `getValues`, `getValueSeparator`, `getValuesList`, `hasArg`, `hasArgName`, `hasArgs`, `hasLongOpt`, `hasOptionalArg`, `hasValueSeparator`, `isDeprecated`, `isRequired`, `isValuesEmpty`, `processValue`, `requiresArg`, many setters, `toDeprecatedString`, `toString`, plus nested `Builder`.
Explanation: The class aggregates identity, argument-count rules, value storage/splitting, type/converter resolution, deprecation metadata, and builder construction, with many getters/setters — a data-and-behavior hub.

Smell: God Class
Code unit: `Options`
File: `org/apache/commons/cli/Options.java`
Evidence: Holds `shortOpts`, `longOpts`, `requiredOpts`, `optionGroups`; methods `addOption` (4 overloads), `addOptionGroup`, `addOptions`, `addRequiredOption`, `getMatchingOptions`, `getOption`, `getOptionGroup`, `getOptionGroups`, `getOptions`, `getRequiredOptions`, `hasLongOption`, `hasOption`, `hasShortOption`, `helpOptions`, `toString`.
Explanation: It simultaneously manages short-option and long-option maps, required-option bookkeeping, option-group registration, and matching, coupling it tightly to `Option` and `OptionGroup`.

Smell: God Class
Code unit: `TextHelpAppendable`
File: `org/apache/commons/cli/help/TextHelpAppendable.java`
Evidence: Static `indexOfWrap`; `adjustTableFormat`, `appendHeader`, `appendList`, `appendParagraph`, `appendTable`, `appendTitle`, `getIndent`, `getLeftPad`, `getMaxWidth`, `getTextStyleBuilder`, `makeColumnQueue`, `makeColumnQueues`, `printQueue`, `printWrapped` (2), `resize` (2), `setIndent`, `setLeftPad`, `setMaxWidth`, `writeColumnQueues`, plus many constants.
Explanation: It handles wrapping, column queue construction, table column resizing/scaling, header/title/list rendering, padding, and output — many distinct formatting responsibilities in one class.

Smell: God Class
Code unit: `TypeHandler`
File: `org/apache/commons/cli/TypeHandler.java`
Evidence: Static `createClass`, `createDate`, `createDefaultMap`, `createFile`, `createFiles`, `createNumber`, `createObject`, `createURL`, `createValue` (3), `createValueUnchecked`,

`getDefault`, `openFile`, `putDefaultMap` (populating ~20 converters), plus instance `getConverter` and the `converterMap` field.

Explanation: It maintains a large converter registry for many types, exposes static factory helpers, and performs conversion lookup — substantial knowledge spread across many types.

Smell: God Class

Code unit: `PatternOptionBuilder`

File: `org/apache/commons/cli/PatternOptionBuilder.java`

Evidence: Many type constants (`STRING_VALUE`, `OBJECT_VALUE`, `NUMBER_VALUE`, `DATE_VALUE`, `CLASS_VALUE`, `EXISTING_FILE_VALUE`, `FILE_VALUE`, `FILES_VALUE`, `URL_VALUE`), `UNSUPPORTED` converter, `getValueClass`, `getValueType` (switch over 9 characters), `isValueCode`, `parsePattern` (builds `Options`, `Option.builder`, `TypeHandler` map, handles `!` required flag), `unsupported`.

Explanation: The class knows about type mapping, pattern-character semantics, option construction, converter registration, and parsing — combining several responsibilities.

3. Long Parameter List

Smell: Long Parameter List

Code unit:

```
HelpFormatter.printHelp(PrintWriter, int, String, String, Options, int, int, String, boolean)
```

File: `org/apache/commons/cli/HelpFormatter.java`

Evidence: Parameter list:

```
(final PrintWriter pw, final int width, final String cmdLineSyntax, final String header, final Options options, final int leftPad, final int descPad, final String footer, final boolean autoUsage)
```

— nine parameters.

Explanation: Nine parameters exceed a comfortable threshold; width, leftPad, descPad, and autoUsage are all formatting-configuration knobs that inflate the signature.

Smell: Long Parameter List

Code unit:

```
HelpFormatter.printHelp(PrintWriter, int, String, String, Options, int, int, String)
```

File: `org/apache/commons/cli/HelpFormatter.java`

Evidence: Parameter list:

`(final PrintWriter pw, final int width, final String cmdLineSyntax, final String header, final Options options, final int leftPad, final int descPad, final String footer)`

— eight parameters.

Explanation: Eight parameters including four separate formatting settings make the signature unwieldy.

Smell: Long Parameter List

Code unit: `HelpFormatter.printHelp(int, String, String, Options, String, boolean)`

File: `org/apache/commons/cli/HelpFormatter.java`

Evidence: Parameter list:

`(final int width, final String cmdLineSyntax, final String header, final Options options, final String footer, final boolean autoUsage)`

— six parameters.

Explanation: Six parameters mixing width, syntax, header, options, footer, and autoUsage.

Smell: Long Parameter List

Code unit: `DefaultParser.parse(Options, Properties, NonOptionAction, String...)`

File: `org/apache/commons/cli/DefaultParser.java`

Evidence: Parameter list:

`(final Options options, final Properties properties, final NonOptionAction nonOptionAction, final String... arguments)`

— four parameters, with two configuration inputs (`properties` , `nonOptionAction`) beyond the core options/arguments pair.

Explanation: Four parameters is borderline, but combining options, properties, a non-option policy enum, and varargs arguments makes the signature carry multiple concerns.

No additional confident Long Parameter List detections were found beyond these; most other methods stay at three or fewer parameters (e.g., `Option` builder methods, `Options.addOption(String, String, boolean, String)` at four parameters is borderline but widely conventional, and `PatternOptionBuilder.parsePattern(String)` is single-parameter).